G.L. Bajaj Institute of Technology and Management Greater Noida

Department of Computer Science & Engineering (Artificial Intelligence and Machine Learning)

Lab File

(BCS - 553)

Design and Analysis of Algorithm

Submitted To:

Ms.Shruti sharma

Department of CSE(AIML)

Submitted By:

Roll No.: 2401921649002

Name: Mayank

LINEAR SEARCH

Linear Search (also called sequential search) is the simplest searching algorithm.

It checks each element of the list/array one by one until it finds the target element (or determines that the element is not in the list).

How it works:

1. Start from the first element of the array.
2. Compare it with the target value.
3. If it matches → return the index.
4. If not → move to the next element.
5. Repeat until the element is found or the end of the array is reached.

Time Complexity:

- **Best Case: O(1) →** if the element is the first one.
- **Worst Case: O(n) →** if the element is last or not present.
- **Average Case: O(n) →** on average, half the elements are checked.

Code:

```
def Linear_Search(arr, target, index=0):
        if index >= len(arr):
            return -1

        if arr[index] == target:
            return index
        return Linear_Search(arr, target, index + 1)
```

```
numbers = [64, 25, 12, 22, 11, 90, 5]
print("Array:", numbers)
print()

test_targets = [22, 5, 64, 100]
for target in test_targets:
    result = Linear_Search(numbers, target)

    print(f"Searching for {target}:")
    print(f"index: {result}")
    if result != -1:
        print(f"Found at index {result}")
    else:
        print(f"Not found")
    print()
```

Output:

```
Array: [64, 25, 12, 22, 11, 90, 5]

Searching for 22:
index: 3
Found at index 3

Searching for 5:
index: 6
Found at index 6

Searching for 64:
index: 0
Found at index 0

Searching for 100:
index: -1
Not found
```

BINARY SEARCH

Binary Search is an efficient searching algorithm used on sorted arrays/lists.

It works by repeatedly dividing the search interval in half until the target element is found or the interval is empty.

How it works:

1. Start with the entire sorted array.
2. Find the **middle element**.
3. If the middle element == target → return its index.
4. If target < middle element → search in the **left half**.
5. If target > middle element → search in the **right half**.
6. Repeat until the element is found or the range is empty.

Time Complexity:

- **Best Case:** O(1) → if middle element is the target.
- **Worst Case:** O(log n) → because the search space halves every step.
- **Average Case:** O(log n)

Code:

```
def Binary_Search(arr, target, left=0, right=None):

    if right is None:
        right = len(arr) - 1
    if left > right:
        return -1
```

```
    mid = left + (right - left) // 2

    if arr[mid] == target:
        return mid

    elif arr[mid] > target:
        return Binary_Search(arr, target, left, mid - 1)

    else:
        return Binary_Search(arr, target, mid + 1, right)

numbers = [64, 25, 12, 22, 11, 90, 5]

print("Original Array:", numbers)

numbers.sort()
print("Sorted Array:", numbers)
print()

test_targets = [22, 5, 64, 100]

for target in test_targets:
    result = Binary_Search(numbers, target)
    print(f"Searching for {target}:")
    print(f"index: {result}")

    if result != -1:
```

```
        print(f"Found at index {result}")
    else:
        print(f"Not found")
    print()
```

Output:

```
Original Array: [64, 25, 12, 22, 11, 90, 5]
Sorted Array: [5, 11, 12, 22, 25, 64, 90]

Searching for 22:
index: 3
Found at index 3

Searching for 5:
index: 0
Found at index 0

Searching for 64:
index: 5
Found at index 5

Searching for 100:
index: -1
Not found
```

INSERTION SORT

Insertion Sort is a simple comparison-based sorting algorithm.

It builds the final sorted array one element at a time by inserting each element into its correct position in the already sorted part of the array.

How it works:

1. Start from the **second element** (first is considered sorted).
2. Compare it with elements before it.
3. Shift larger elements one position to the right.
4. Insert the element into the correct position.
5. Repeat until the entire array is sorted.

Time Complexity:

- **Best Case:** O(n) (already sorted array, only 1 comparison per element).
- **Worst Case:** $O(n^2)$ (reverse sorted array).
- **Average Case:** $O(n^2)$

Code:

```
def Inserstion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
```

```
            arr[j + 1] = key


    if __name__ == "__main__":
        arr = [9,5,4,15,7,-1,-4,-6]
        print(f"Original Array: {arr}")

        Inserstion_sort(arr)
        print(f"\nSorted Array: {arr}")
```

Output:

```
Original Array: [9, 5, 4, 15, 7, -1, -4, -6]

Sorted Array: [-6, -4, -1, 4, 5, 7, 9, 15]
```

SELECTION SORT

Selection Sort is a simple comparison-based sorting algorithm.

It works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the array and putting it in its correct position.

It always takes quadratic time because it always scans the remaining elements, even if already sorted.

How it works:

1. Start with the first position.
2. Find the **smallest element** in the unsorted part of the array.
3. Swap it with the element at the current position.
4. Move to the next position and repeat until the entire array is sorted.

Time Complexity:

- **Best Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Average Case:** $O(n^2)$

Code:

```
def Selection_Sort(arr):
    n = len(arr)

    for i in range(n):
       min_index = i
```

```
            for j in range(i + 1, n):
                if arr[j] < arr[min_index]:
                    min_index = j

            arr[i], arr[min_index] = arr[min_index], arr[i]

    arr = [-12, -13, -65, -1, 0, 1, 3, 5, 10]
    print(f"Unsorted Array:{arr}")
    Sorted_Array = Selection_Sort(arr)
    print(f"\nSorted Array: {arr}")
```

Output:

```
Unsorted Array:[-12, -13, -65, -1, 0, 1, 3, 5, 10]

Sorted Array: [-65, -13, -12, -1, 0, 1, 3, 5, 10]
```

QUICK SORT

Quick Sort is a divide-and-conquer sorting algorithm.

It works by selecting a pivot element, partitioning the array around the pivot (smaller on left, larger on right), and then recursively sorting the left and right subarrays.

How it works:

1. Choose a **pivot** element (commonly first, last, or random).
2. Rearrange elements so that:
    - Elements smaller than pivot go to the **left**
    - Elements greater than pivot go to the **right**
3. Recursively apply Quick Sort to the left and right subarrays.
4. Combine results → sorted array.

Time Complexity:

- **Best Case:** O(n log n) (pivot splits array evenly).
- **Average Case:** O(n log n)
- **Worst Case:** O(n^2) (bad pivot choice, e.g., always choosing first element in a sorted array).

Code:

```
def Quick_Search(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
```

```
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return Quick_Search(left) + middle + Quick_Search(right)

arr = [-12, -13, -65, -1, 0, 1, 3, 5, 10]
Sorted_array = Quick_Search(arr)
print(f"Unsorted Array: {arr}")
print(f"\nSorted Array: {Sorted_array}")
```

Output:

```
Unsorted Array: [-12, -13, -65, -1, 0, 1, 3, 5, 10]

Sorted Array: [-65, -13, -12, -1, 0, 1, 3, 5, 10]
```

MERGE SORT

Merge Sort is a divide-and-conquer sorting algorithm.

It works by dividing the array into halves, sorting each half recursively, and then merging the sorted halves into a single sorted array.

How it works:

1. If the array has 1 or 0 elements → it's already sorted.
2. Divide the array into two halves.
3. Recursively apply merge sort to each half.
4. Merge the two sorted halves into a single sorted array.

Time Complexity:

- **Best Case:** O(n log n)
- **Average Case:** O(n log n)
- **Worst Case:** O(n log n) (always guaranteed)

Code:

```
def Merge_Sort(arr):
  if len(arr) <= 1:
     return arr

  mid = len(arr) // 2
  left_half = Merge_Sort(arr[:mid])
  right_half = Merge_Sort(arr[mid:])

  return merge(left_half, right_half)
```

```
    def merge(left, right):
        merged = []
        i = j = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                merged.append(left[i])
                i += 1

            else:
                merged.append(right[j])
                j += 1

        merged.extend(left[i:])
        merged.extend(right[j:])

        return merged

    arr = [-12, -13, -65, -1, 0, 10, 1, 87, 23]
    print(f"Unsorted Array: {arr}")
    Sorted_Array = Merge_Sort(arr)
    print(f"\nSorted Array: {Sorted_Array}")
```

Output:

```
Unsorted Array: [-12, -13, -65, -1, 0, 10, 1, 87, 23]

Sorted Array: [-65, -13, -12, -1, 0, 1, 10, 23, 87]
```

Knapsack Problem Using the Greedy Algorithm

You are given:

- A bag (knapsack) with a maximum weight capacity W
- n items, each with:
    - value ***v[i]***
    - weight ***w[i]***

Goal:

Maximize total value you can put in the bag.

But in Fractional Knapsack,

You CAN take fractions of an item (cut it).

Greedy Strategy:

1. Compute value per unit weight for each item:

$$\textbf{ratio}_i = \frac{v[i]}{w[i]}$$

2. Sort items in descending order of ratio.
3. Pick items until the bag is full:
    - If full item fits → take all
    - Else → take fraction of the item and stop

This greedy approach gives the optimal solution.

Time Complexity:

- Sorting items: **O(n log n)**
- Selecting items: **O(n)**
  Total = O(n log n)

Code:

```
    def fractional_knapsack(values, weights, capacity):
        items = list(zip(values, weights))

        # Step 1: Sort by value/weight ratio
        items.sort(key=lambda x: x[0] / x[1], reverse=True)

        total_value = 0
        remaining = capacity

        for value, weight in items:
            if remaining == 0:
                break

            if weight <= remaining:
                # take the whole item
                total_value += value
                remaining -= weight
            else:
                # take fraction
                fraction = remaining / weight
                total_value += value * fraction
                remaining = 0

        return total_value



# Example
```

```
values = [60, 100, 120]
weights = [10, 20, 30]
capacity = 50

print("Maximum value:", fractional_knapsack(values, weights, capacity))
```

Output:

```
Maximum value: 240.0
```

Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted graph

Kruskal's algorithm finds a **Minimum Spanning Tree (MST)** of a connected, weighted graph by repeatedly adding the smallest-weight edge that doesn't create a cycle. It uses a **Disjoint Set Union (DSU) / Union-Find data structure** to detect cycles efficiently.

How it works:

1. Sort all edges by weight (ascending).
2. Iterate edges from smallest to largest.
3. If an edge connects two different components (different DSU roots), add it to the MST and union their sets.
4. Stop when MST has ***V-1*** edges (V = number of vertices).

Time Complexity:

- Sorting edges: **O(E log E)** (dominant).
- Union-Find operations with path compression & union by rank: near **O(α(V))** per op (practically constant).
  Overall: **O(E log E)** ≈ **O(E log V)**.
  Space: **O(V + E)**.

Code:

```
class DisjointSet:
  def __init__(self, n):
    # parent[i] = parent of node i; initially parent[i] = i
    self.parent = list(range(n))
    self.rank = [0] * n
```

```
    def find(self, x):
        # path compression
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # union by rank
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False  # already in same set
        if self.rank[rx] < self.rank[ry]:
            self.parent[rx] = ry
        elif self.rank[rx] > self.rank[ry]:
            self.parent[ry] = rx
        else:
            self.parent[ry] = rx
            self.rank[rx] += 1
        return True

def kruskal_mst(num_vertices, edges):
    """
    num_vertices: number of vertices (vertices numbered 0..num_vertices-1)
    edges: list of tuples (weight, u, v)
    Returns: (mst_weight, mst_edges) where mst_edges is list of (u, v, weight)
    """
    # sort edges by weight
    edges_sorted = sorted(edges, key=lambda e: e[0])
```

```
    dsu = DisjointSet(num_vertices)

    mst_edges = []
    mst_weight = 0

    for w, u, v in edges_sorted:
        if dsu.union(u, v):
            mst_edges.append((u, v, w))
            mst_weight += w
            if len(mst_edges) == num_vertices - 1:
                break

    # If graph wasn't connected, we won't have V-1 edges
    if len(mst_edges) != num_vertices - 1:
        raise ValueError("Graph is not connected — MST does not exist.")

    return mst_weight, mst_edges

# Example usage
if __name__ == "__main__":
    # Graph: 4 vertices (0..3), edges given as (weight, u, v)
    edges = [
        (1, 0, 1),
        (3, 0, 2),
        (4, 0, 3),
        (2, 1, 2),
        (5, 1, 3),
        (6, 2, 3)
```

```
]
w, mst = kruskal_mst(4, edges)
print("MST total weight:", w)
print("MST edges:")
for u, v, wt in mst:
    print(f"{u} -- {v} : {wt}")
```

Output:

```
MST total weight: 7
MST edges:
0 -- 1 : 1
1 -- 2 : 2
0 -- 3 : 4
```

N Queens on an N*N Chessboard such that no two queens attack each other using the Backtracking technique

Place ***N*** queens on an ***N×N*** chessboard so that no two queens attack each other (no two share same row, column, or diagonal).
Backtracking tries placing a queen row-by-row and **backs up** when a conflict occurs.

Core idea (backtracking):

1. Place one queen per row (so rows are implicitly handled by recursion depth).
2. For the current row, try each column ***c*** from ***0..N-1***.
3. If column ***c*** and both diagonals are free, place queen and recurse to next row.
4. If you reach row ***N***, you found a valid solution.
5. Otherwise remove the queen (backtrack) and try next column.

Use sets (or boolean arrays) to check conflicts in O(1):

- ***cols*** — columns already taken
- ***diag1*** — major diagonals (r - c)
- ***diag2*** — minor diagonals (r + c)

Time & Space Complexity:

- **Time:** exponential. Often approximated as ***O(N!)*** in worst case — backtracking prunes a lot, but there is no known polynomial-time algorithm.
- **Space:** ***O(N)*** extra (for cols/diags + recursion stack) plus ***O(N^2)*** if storing all solutions.

Code:

```
def count_n_queens_bitmask(N):
    """
```

```
    Return the number of distinct solutions for N-Queens using bitmask backtracking.
    Works well up to N ~ 14-15 in reasonable time in Python.
    """
    def backtrack(columns, diag_left, diag_right):
        # columns, diag_left, diag_right are bitmasks of occupied positions
        if columns == all_ones:
            return 1
        count = 0
        # positions available this row
        free = all_ones & ~(columns | diag_left | diag_right)
        while free:
            # pick the rightmost free bit
            p = free & -free
            free -= p
            count += backtrack(columns | p, (diag_left | p) << 1, (diag_right | p) >> 1)
        return count

    all_ones = (1 << N) - 1
    return backtrack(0, 0, 0)

# Example usage:
if __name__ == "__main__":
    for n in range(1, 9):
        print(f"N={n}, solutions={count_n_queens_bitmask(n)}")
```

Output:

```
…   N=1, solutions=1
    N=2, solutions=0
    N=3, solutions=0
    N=4, solutions=2
    N=5, solutions=10
    N=6, solutions=4
    N=7, solutions=40
    N=8, solutions=92
```